PROJECT REVIEW / STATIC SOURCE MAP

CleanCare

Code map and authentication assessment

Frontend and backend files, function responsibilities, feature workflows, data flow, and security findings.

Review date	29 October 2026
Repository	clean-care / local checkout
Review method	Read-only static review; no code changes

System path

React browser app -> Axios -> Express routes -> role middleware where mounted -> controllers -> Mongoose -> MongoDB.

Assessment

High risk for deployment. A public endpoint can create a superAdmin; many user and job APIs are unprotected; a mail credential is hard-coded in a legacy controller. Details and remediation order are in the report.

Credentials and environment-file values are omitted. No application source was changed.

Path	Purpose / function map
README.md; frontend/README.md	Root README has only a heading; frontend README is empty.
frontend/package.json	Create React App scripts and UI dependencies. Scripts: start, build, test, eject.
frontend/package-lock.json	Pinned dependency resolution for the frontend.
frontend/.gitignore	Ignores node_modules, build, coverage, local env variants, and editor/log files.
frontend/htaccess	Apache rewrite rules route non-file paths to index.html for SPA routing.
frontend/public/index.html	HTML mount point, page title/metadata, manifest and CDN Bootstrap includes.
frontend/public/manifest.json	Installable web app metadata.
frontend/public/robots.txt	Crawler instruction file.
frontend/public/favicon.svg, favicon.ico	Browser tab icons.
frontend/public/logo.png, logo.ico, assets/*	Static logo, favicon, and user image assets.
frontend/build/**	Generated CRA production bundle: HTML, manifest, icons, copied assets, hashed CSS/JS chunks, and source maps. Treat as output, not the working source of features.

Frontend runtime and shared files

Path	Purpose and named functions
frontend/src/index.js	React bootstrap: creates root, loads global CSS, wraps App in StrictMode and BrowserRouter.
frontend/src/App.js	App() owns theme/sidebar state, hides chrome on /, and maps URL paths to scenes and allowed roles.
frontend/src/PrivateRoutes.js	PrivateRoute(children, allowedRoles) decodes token, checks exp and role, stores User_role, and redirects.
frontend/src/Utils/PrivateRoutes.jsx	Older PrivateRoutes() checks isAuthenticated and renders Outlet; not referenced by App.js.
frontend/src/environment.js	environment.apiUrl hardcodes the local API base URL.
frontend/src/ApiConfig.jsx	Legacy Axios clients and hosted content API wrappers; no imports from active scenes were found.
frontend/src/theme.js	tokens(mode) supplies compatibility color tokens; themeSettings(mode) builds MUI theme; ColorModeContext exposes toggle; useMode() persists light/dark mode.
frontend/src/styles/uiStyles.js	Shared MUI style objects: pageContainerSx, pageHeaderRowSx, surfaceSx, tableSurfaceSx, filterBarSx, formPageSx, formSurfaceSx, actionButtonSx.
frontend/src/index.css	Global layout, typography smoothing, link/image defaults, scrollbar styling, and responsive overflow.
frontend/src/scenes/login/login.css	Login-page layout and responsive styles.
frontend/src/scenes/global/Sidebar.jsx	NavItem() renders a link; Sidebar() renders responsive role-sensitive navigation; handleNavigate() closes mobile menu; matches() checks active route.
frontend/src/scenes/global/Topbar.jsx	formatRole() labels roles; Topbar() loads user display data; fetchAdminData() reads the signed-in user; handleLogout() confirms and clears local storage.
frontend/src/components/Header.jsx	Header(title, subtitle) renders page heading.
frontend/src/components/Brand.jsx	Brand(compact, inverse) renders the CleanCare mark and wordmark.
frontend/src/components/DetailGrid.jsx	formatFieldLabel() humanizes keys; renderDefaultValue() handles empty, boolean, nested, and status values; DetailGrid() renders key/value details.
frontend/src/components/SweetAlert.jsx	SweetAlert() configures a confirmation dialog; handleConfirmClick() calls the provided confirmation callback.

Frontend data and static assets

Path	Purpose
frontend/src/data/countries.js	Static country list; no active imports found.
frontend/src/data/mockData.js	Fixture exports for generic team, event, content, contact, sermon, template, membership, feedback, invoice, transactions, chart, geography, media, blog, artist, and service data. No active imports found.
frontend/src/data/mockGeoFeatures.js	Large geographic feature/geometry dataset; no active imports found.
frontend/src/assets/background.jpg	Login artwork.
frontend/src/assets/logo/*	Profile and logo image assets.

frontend/public/assets/*	Public copies of profile, logo, and favicon assets.
frontend/src/scenes/profile/index.jsx	Older Profile() screen fetches profile data and renders a summary; App routes to EditProfile instead, so this component is not wired into the current route tree.

Route map and access intended by the frontend

React routes

Path	Scene	Client-side allowed roles
/	Login	Public
/dashboard	Dashboard	superAdmin, admin
/jobs	Jobs	superAdmin, admin, staff
/jobs/newjob	AddJob	superAdmin, admin
/jobs/viewjob/:id	ViewJob	superAdmin, admin, staff
/jobs/editjob/:id	EditJob	superAdmin, admin
/completedjobs	CompletedJobs	superAdmin, admin
/paymentsummery	PaymentSummery	superAdmin, admin
/clients and /clients/newclient	Clients, AddClient	superAdmin, admin
/clients/viewclient/:id	ViewClient	superAdmin, admin
/clients/editclient/:id	EditClient	superAdmin, admin
/clients/viewclient/:id/jobs	ClientJobs	superAdmin, admin
/staff and /staff/newstaff	Staff, AddStaff	superAdmin, admin
/staff/viewstaff/:id	ViewStaff	superAdmin, admin
/staff/editstaff/:id	EditStaff	superAdmin, admin
/staff/viewstaff/:id/jobs	StaffJobs	superAdmin, admin
/invoices and /invoices/newInvoice	Invoices, AddInvoice	superAdmin, admin
/invoices/editInvoice/:id and /invoices/viewInvoice/:id	EditInvoice, ViewInvoice	superAdmin, admin
/profile and /profile/changepassword	EditProfile, ChangePw	superAdmin, admin, staff
/admin and /admin/createadmin	Admin, CreateAdmin	superAdmin, admin
/admin/viewadmin/:id and /admin/editadmin/:id	ViewAdmin, EditAdmin	superAdmin, admin

These are browser-side guards. The server route matrix later in this report is the source of actual API enforcement. A hidden menu item or a rejected React route does not secure an API endpoint.

Frontend feature files and functions

Jobs and payments UI

File	Functions / behavior
scenes/jobs/index.jsx	Jobs() lists and filters jobs; getUserIdFromToken() and getUserRoleFromToken() read JWT claims; formatTime() formats timestamps; fetchJobs() selects all-job or staff-job API and applies date/staff/client filters; fetchStaffAndClients() loads filter options; exportToPdf() exports rows; handleStatusChange() patches jobStatus; handleDeleteClick() deletes after confirmation; handleEditClick() is empty; handleViewJobs() refreshes; vievAlljobs() clears filters; handleSignInSignOut() calls time tracking API; createSignInOutColumn() selects staff action/status column.
scenes/jobs/AddJob.jsx	AddJob() builds assignment form; fetchClients() and fetchStaffs() load active choices; handleFormSubmit() posts a job; jobSchema and initialValues define validation/defaults.
scenes/jobs/EditJob.jsx	EditJob() loads active client/staff choices and selected job; fetchClients(), fetchStaffs(), fetchJobDetails() populate the form; submit handler updates job; jobSchema validates editable fields.
scenes/jobs/ViewJob.jsx	ViewJob() loads a job; getUserRoleFromToken() hides organization-side amounts for staff; fetchJobDetails() formats populated client/staff for DetailGrid.

File	Functions / behavior
scenes/jobs/completedJobs/index.jsx	CompletedJobs() fetches completed jobs; exportToPdf() exports the visible set.
scenes/jobs/paymentSummery/index.jsx	Jobs() renders payment summary; token helpers read user ID/role; formatTime() formats dates; fetchJobs() loads/filter job data; calculateSummary() totals organization, staff, and gross amounts; fetchStaffAndClients() loads filter choices; exportToPdf() exports; handleStatusChange() changes job status; handleDeleteClick() deletes; handleEditClick() is empty; handleViewJobs() refreshes; vievAlljobs() clears filters.

Client, staff, admin, invoice, and profile UI

File	Functions / behavior
scenes/clients/index.jsx	Clients() lists clients; fetchClients() maps API records to grid rows; exportToPdf() exports; handleDeleteClick() deletes; handleStatusChange() patches clientStatus; handleEditClick() is empty (edit navigation is primarily provided by row links/actions).
scenes/clients/AddClient.jsx	AddClient() renders a Formik form; handleFormSubmit() posts values; clientSchema and initialValues define validation/defaults.
scenes/clients/EditClient.jsx	EditClient() loads a client; fetchClientDetails() fills values; Yup schema validates; useFormik submit updates it.
scenes/clients/ViewClient.jsx	ViewClient() fetches and formats one client for DetailGrid.
scenes/clients/ClientJobs .jsx	ClientJobs() lists a client's jobs; fetchClientJobs() calls filtered jobs endpoint; handleViewJobs() refreshes; calculateTotalPayments() sums job payment values.
scenes/staff/index.jsx	Staff() lists staff; fetchStaff() loads rows; exportToPdf() exports; handleDeleteClick() deletes; handleStatusChange() patches workStatus; handleEditClick() is empty.
scenes/staff/AddStaff.jsx	AddStaff() gathers staff and login fields; handleFormSubmit() posts /user/addUser with role=staff and nested staffDetails.
scenes/staff/EditStaff.jsx	EditStaff() fetches a staff record; fetchStaffDetails() initializes form; handleUpdateStaff() submits profile values through /user/updateStaff.
scenes/staff/ViewStaff.jsx	ViewStaff() loads staff and linked user; fetchStaffDetails() fetches staff record; fetchUserEmail() fetches linked account data.
scenes/staff/StaffJobs.jsx	StaffJobs() lists jobs for a staff ID; fetchStaffJobs() loads jobs; calculateTotalPayments() totals payments; handlePaymentStatusChange() writes payment status; handleViewJobs() refreshes.
scenes/admins/index.jsx	Admin() lists user accounts with admin role; fetches /admin/getAllAdmins and supports row actions/status flow.
scenes/admins/CreateAdmin.jsx	CreateAdmin() builds admin/user form; handleClickShowPassword() toggles password visibility; handleMouseDownPassword() prevents focus loss; handleFormSubmit() posts /user/addUser with role=admin and adminDetails.
scenes/admins/EditAdmin.jsx	EditAdmin() loads user/admin data; fetchAdminDetails() hydrates form; handlePasswordChange() updates password fields; three visibility handlers toggle password visibility; handleMouseDownPassword() preserves focus; validatePassword() checks strength; handleUpdateAdmin() updates admin/profile; handleUpdatePassword() changes password.
scenes/admins/ViewAdmin.jsx	ViewAdmin() fetches a user and populated adminDetails for DetailGrid.
scenes/invoices/index.jsx	Invoices() fetches and maps invoices; exportToPdf() exports; handleDeleteClick() deletes; handleEditClick() navigates; handleStatusChange() patches invoiceStatus; handleViewInvoices() refreshes; columns configure grid rendering.
scenes/invoices/AddInvoices.jsx	AddInvoice() loads active clients; fetchClients() loads choices; handleFormSubmit() posts invoice values; invoiceSchema and initialValues define form rules.
scenes/invoices/EditInvoices.jsx	EditInvoice() loads client choices and one invoice; fetchClients() and fetchInvoiceDetails() hydrate form; handleUpdateInvoice() saves changes; invoiceSchema validates.
scenes/invoices/ViewInvoice.jsx	ViewInvoice() fetches one invoice and formats its populated client for DetailGrid.
scenes/profile/EditProfile.jsx	EditProfile() derives user ID from JWT, fetches account data, submits name/email changes, handles file selection and preview, uploads an image to an external hosted API, and removes/resets the image. Named helpers: getUserIdFromToken(), fetchUserDetails(), handleFileChange(), uploadProfileImage(), removeProfileImage(), handleRemoveImage(), handleInputChange().
scenes/profile/ChangePw.jsx	CreateNewUser() is the exported component name for the password-change screen; getUserIdFromToken() reads the JWT subject; handleFormSubmit() calls /user/updateUserPassword; visibility handlers show/hide current/new/confirm values; Yup schema validates.
scenes/profile/index.jsx	Profile() is a legacy read-only profile card; getAdminData() loads data; it is not mounted in App.js.

Legacy frontend API adapter and non-feature code

frontend/src/ApiConfig.jsx constructs API, multipartAPI, and a default ApiConfig client against a fixed hosted API base. It reads localStorage token once at module load. No current scene imports this module, so the wrappers below are not part of

the active CleanCare UI flow.

ApiConfig function groups

Functions	Purpose
getAllCareers, deleteCareer	Read or delete career entries.
getAllContacts, getContact, deleteContact	List, read one, and delete contacts.
getAllNews, createNews, deleteNewsById	List, create, and delete news.
getAllForms	Read submitted forms.
formatDate, formatDateToReadable	Convert API date strings into display dates.
getAllAudio, getAudio, createAudio, updateAudio, deleteAudio	Audio content CRUD.
getAllArtists, getArtistByID, createArtist, updateArtist, updateArtistStatusById, deleteArtist	Artist CRUD and status update.
createWorshioService, getAllWorshipService, getWorshipServiceByID, updateWorshipService, deleteWorshipService	Worship service CRUD; create function spelling is preserved from source.
getAllBlogs, getBlogByID, createBlog, updateBlog, updateBlogStatusById, deleteBlog	Blog CRUD and status update.
getAllVideo, getVideo, createVideo, updateVideo, deleteVideo	Video content CRUD.
getAllWorshipCenters, getWorshipCenter, createWorshipCenter, updateWorshipCenter, deleteWorshipCenter	Worship center CRUD.
getAllSermon, getSermon, createSermon, updateSermon, deleteSermon	Sermon CRUD.
getAllContent, getContent, createContent, updateContent, deleteContent	Generic page/content CRUD.
getAllEvent, getEvent, createEvent, updateEvent, deleteEvent	Event CRUD.
getAllCourse, getCourse, createCourse, updateCourse, deleteCourse	Course CRUD.
getAllTemplates, getTemplateByID, createTemplate, updateTemplate, updateTemplateStatusById, deleteTemplates	Template CRUD and status update.
getAllFeedbacks, getFeedbackByID, createFeedback, deleteFeedbacks	Feedback list/read/create/delete.
getAllMembershipCards, getMembershipCardByID, createMembershipCard, updateMembershipCard, updateMembershipCardStatusById, deleteMembershipCards	Membership card CRUD and status update.

The mock datasets in frontend/src/data/mockData.js and mockGeoFeatures.js are sample chart/content/geography payloads. The active CleanCare scenes do not import them. frontend/src/data/countries.js is also not imported. The public and src asset folders include duplicated branding and profile assets.

Other static files: frontend/public/robots.txt and manifest.json are deployment/browser metadata; frontend/public/index.html is the CRA shell; frontend/htaccess enables history fallback. frontend/build/** is compiled output. backend/.gitignore excludes local dependencies/logs. backend/package.json starts index.js through nodemon; its test script is a placeholder that exits with failure.

Backend route and endpoint map

Routes are mounted beneath /api in backend/index.js. 'Protected' below means the route installs roleAuthentication with the listed allowed role; 'Public' means no middleware is attached in its route file.

Admin, user, and staff routes

Endpoint	Handler	API access
POST /admin/register	registerAdmin	Public
POST /admin/login	loginAdmin	Public
PUT /admin/updateAdminPassword/:id	updateAdminPassword	Public
GET /admin/getAllAdmins	getAllAdmins	Public
PATCH /admin/updateStatus/:id	updateStatus	superAdmin
POST /user/addUser	addUser	superAdmin, admin
POST /user/addsuperAdmin	addsuperAdmin	Public
POST /user/loginUser	loginUser	Public
GET /user/getUser/:id	getUser	Public

Endpoint	Handler	API access
GET /user/getAllUsers	getAllUsers	Public
DELETE /user/deleteUser/:id	deleteUser	Public
PUT /user/updateUserPassword/:id	updateUserPassword	Public
PUT /user/updateStaff/:id	updateStaff	Public
PUT /user/userUpdate/:id	updateUser	Public
PUT /user/updateAdmin/:id	updateAdmin	Public
POST /staff/addStaff	addStaffMember	superAdmin, admin
PUT /staff/updateStaffById/:id	updateStaffById	superAdmin, admin
DELETE /staff/deleteStaffById/:id	deleteStaffById	superAdmin, admin
GET /staff/getStaffById/:id	getStaffById	superAdmin, admin
GET /staff/getAllStaffsPagination	getAllStaffsPagination	superAdmin, admin
GET /staff/getAllActiveStaff	getAllActiveStaff	superAdmin, admin
GET /staff/getAllStaff	getAllStaff	superAdmin, admin
PATCH /staff/updateStatus/:id	updateStatus	superAdmin, admin
GET /staff/getAllStaffCount	getAllStaffCount	Public
POST /staff/loginStaff	loginStaff	Public

Client and job routes

Endpoint	Handler	API access
POST /client/addClient	addClient	superAdmin, admin
PUT /client/updateClientById/:id	updateClientById	superAdmin, admin
DELETE /client/deleteClientById/:id	deleteClientById	superAdmin, admin
GET /client/getClientById/:id	getClientById	superAdmin, admin
GET /client/getAllClientPagination	getAllClientPagination	superAdmin, admin
GET /client/getAllActiveClient	getAllActiveClient	superAdmin, admin
GET /client/getAllClient	getAllClient	superAdmin, admin
PATCH /client/updateStatus/:id	updateStatus	superAdmin, admin
GET /client/getAllClientCount	getAllClientCount	Public
POST /job/addJob	addJob	superAdmin, admin
PUT /job/updatedJob/:id	updatedJob	superAdmin, admin
DELETE /job/deleteJobById/:id	deleteJobById	Public
GET /job/getJobById/:id	getJobById	Public
GET /job/getAllJobPagination	getAllJobPagination	Public
GET /job/getAllJobs	getAllJobs	Public
GET /job/getCount	getAllJobsCount	Public
GET /job/getTotalAndProfit	getTotalAndProfit	Public
GET /job/getAllCompletedJobs	getAllCompletedJobs	Public
PATCH /job/updateStatus/:id	updateStatus	Public
PUT /job/signInJob/:jobId	signInJob	Public
PUT /job/signOffJob/:jobId	signOffJob	Public
GET /job/getInvoice/:jobId	getInvoice	Public
GET /job/getJobsbyStaff/:staffId	getJobsByStaffId	Public
GET /job/getFilteredJobs/:clientId	getFilteredJobs	Public
GET /job/getStaffJobsbyId/:assignedStaff	getStaffJobsbyId	Public
PUT /job/paymentJob	paymentJob	Public

Invoice routes and server file map

Path	Purpose / function map
POST /invoice/addInvoice	addInvoice; protected for superAdmin/admin.
PUT /invoice/updatedInvoice/:id	updatedInvoice; protected for superAdmin/admin.
DELETE /invoice/deleteInvoiceById/:id	deleteInvoiceById; protected for superAdmin/admin.
GET /invoice/getInvoiceById/:id	getInvoiceById; protected for superAdmin/admin.
GET /invoice/getAllInvoices	getAllInvoices; protected for superAdmin/admin; optional sendDate range.
PATCH /invoice/updateStatus/:id	updateStatus; protected for superAdmin/admin.
backend/index.js	Loads dotenv, Express, body parsers, open CORS, route mounts, Mongo connection, then listens on configured port.
backend/package.json / package-lock.json	Express/Mongoose/JWT/bcrypt/Multer/Nodemailer dependencies, nodemon start; test script is an unimplemented placeholder.
backend/.gitignore	Excludes node_modules, logs, local files.
backend/EmailSend/sendEmail.js	sendEmail(email, htmlMessage) sends HTML email via Nodemailer using USER_EMAIL and USER_PASS env variables.
backend/middleware/roleAuthentication.js	roleAuthentication(allowedRoles) returns Express middleware: verifies bearer JWT, reloads User role, sets req.userId and req.userRole, then continues or returns 401/403.

Backend file and function catalog

Mongoose models

File	Data represented
models/userModel.js	User login principal: userName, hashed password, unique email, role enum admin/staff/superAdmin, active/inactive status, and optional references to adminDetails, staffDetails, superAdminDetails. Timestamps enabled.
models/adminModel.js	Admin profile references User and has firstName, lastName, position. No email, password, or role fields.
models/superAdminModel.js	SuperAdmin profile references User.
models/staffModel.js	Staff profile references User, names, address/phone, position, dates, emergency contact, bank fields, adminId, notes, and workStatus. No email, password, or role fields. bankAcBranch appears twice in the schema.
models/clientModel.js	Client contact/business fields, required email/address/phone/name, required adminId reference, optional notes, clientStatus, timestamps.
models/jobModel.js	Job name/description, Client and Staff refs, client/staff hours/rates/totals, required adminId, notes, payment, paymentStatus, jobStatus, scheduled date, sign-in/out timestamps and booleans.
models/invoiceModel.js	Invoice title/description, sendDate, Client ref, amount, invoiceStatus, timestamps.

User and admin controller functions

Function	What it does
userController.createToken	Signs {_id, role} with SECRET for 259200 seconds (3 days).
addUser	Validates required fields/email, hashes password, creates User, then creates Admin or Staff detail and links it. No transaction; account may be left partially written on failure.
addsuperAdmin	Public route creates a User with superAdmin role and a SuperAdmin detail record; no authorization or duplicate-email precheck.
loginUser	Looks up User by email, compares bcrypt hash, returns a three-day JWT. Does not reject inactive status.
getUser	Loads a user and populated profile records, then returns selected user fields without password.
updateUser	Updates userName/email for any supplied ID.
getAllUsers	Returns all populated User documents; response includes password hashes.
deleteUser	Deletes a User by ID and removes related Admin or Staff record for those roles; no caller check and SuperAdmin cleanup omitted.
updateUserPassword	Requires currentPassword/newPassword, verifies existing hash, then hashes and saves new password for arbitrary user ID.
updateStaff	Updates Staff detail fields and associated User name/email/status for arbitrary Staff ID.

Function	What it does
updateAdmin	Updates Admin profile and linked User name/email for arbitrary Admin ID.
adminController.createToken	Signs {_id, role} from an Admin document; middleware later expects that ID in User collection.
registerAdmin	Admin schema requires User ref and omits email/password/role; bootstrap cannot match it.
loginAdmin	Looks up Admin by email and compares password; emits token using Admin ID and role.
updateAdminPassword	Hashes a new password on a User record ID, without checking current password or authorizing caller.
getAllAdmins	Queries User records with role=admin and populates adminDetails.
adminController.updateStatus	Changes a User status; route is restricted to superAdmin.

Staff and client controller functions

Function	What it does
staffController.createToken	Signs {_id, role} for the legacy Staff login path.
addStaffMember	Legacy Staff-only create path hashes password and builds Staff from request; reads req.adminId.id, which roleAuthentication does not set.
updateStaffById	Finds and replaces editable Staff detail fields.
deleteStaffById	Deletes a Staff profile by ID.
getStaffById	Reads one Staff profile.
getAllStaff	Reads every Staff profile; returns 404 if empty.
getAllStaffCount	Loads all Staff documents and counts them; route is public.
getAllActiveStaff	Reads Staff where workStatus is active.
loginStaff	Legacy login looks for email/password on Staff and signs a token from Staff ID.
generateOTP / sendEmail	Generates a six-digit OTP and sends email through Nodemailer; source includes a hard-coded credential.
requestPasswordReset	Stores OTP and one-hour expiry on a matching Staff document, then emails it. Not mounted in staffRoute.
verifyOTP	Checks supplied OTP and expiry against a Staff record. Not mounted.
resetPassword	Intended to validate OTP and set a hashed password; references PassangerModel, which is not imported/defined. Not mounted.
getAllStaffsPagination	Searches a small set of fields and returns paged Staff records; route is protected.
staffController.updateStatus	Changes workStatus for a Staff record; route is protected.
addClient	Creates Client, checks duplicate email, and assigns owner using req.userId._id.
updateClientById	Replaces client contact/business fields.
deleteClientById	Deletes one Client record.
getClientById / getAllClient	Reads one client or all clients.
getAllClientCount / getAllActiveClient	Counts all clients or reads active clients.
getAllClientPagination	Searches first/last name/address, sorts newest-first, and returns pagination metadata.
clientController.updateStatus	Updates clientStatus.

Job and invoice controller functions

Function	What it does
addJob	Validates Client and Staff refs; calculates orgTotal and staffPayTotal; saves job and owner reference.
updatedJob	Recalculates totals and updates core job fields.
deleteJobById / getJobById	Deletes a job or reads one with populated staff/client.
getAllJobs	Lists jobs with optional date, staff, and client filters; populates names.
getAllJobsCount / getTotalAndProfit	Counts jobs; aggregates organization totals, staff totals, and gross difference.
getAllJobPagination	Searches firstName/lastName/address against Job documents; those fields are not in Job schema, so its search intent does not match stored fields.
jobController.updateStatus / getAllCompletedJobs	Changes jobStatus or returns jobs in Completed state.
signInJob / signOffJob	Writes assigned staff and sign-in time/boolean, then sign-off time/boolean. Routes are public; response uses data, while UI reads response.data.job.

Function	What it does
getInvoice	Builds a completed-job invoice payload. It asks Client for a name field and reads noOfhours/hourRate, which do not match the Client/Job schema field names.
getJobsByStaffId	Maps User ID to Staff profile then lists assigned jobs, optionally by date.
getFilteredJobs	Filters by client/status/month/year/date and returns jobs plus summed payment.
getStaffJobsbyId	Lists jobs assigned to the Staff ID with optional status/date filters.
paymentJob	Sets paymentStatus, but also unconditionally sets jobStatus to Cancelled.
addInvoice	Checks Client exists, then creates Invoice.
updatedInvoice	Updates title, description, date, client, and amount.
getInvoiceById / getAllInvoices	Reads one populated invoice or lists populated invoices with optional sendDate range.
invoiceController.updateStatus / deleteInvoiceById	Changes invoiceStatus or deletes an invoice.

Feature workflows and data flow

Sign-in and session lifecycle

Step	Flow
1. Submit	Login screen validates email shape and requires a password matching at least eight characters, upper/lowercase and digit; posts credentials to /user/loginUser.
2. Verify	loginUser loads User by email and bcrypt-compares password. On success it signs {_id, role} for three days and returns token.
3. Store	Browser stores token and isAuthenticated marker in localStorage, then navigates to /dashboard.
4. Guard	PrivateRoute decodes JWT locally, compares exp with current time, stores User_role, checks allowedRoles. This is a presentation guard, not signature verification.
5. API authorization	Protected endpoints should receive Bearer token. roleAuthentication verifies signature with SECRET and reloads User role from MongoDB.
6. Logout	Topbar asks for confirmation, clears all localStorage, and navigates to /; no server-side token revocation occurs.

The UI's login path uses User. The /admin/login and /staff/loginStaff endpoints represent older parallel identity stores and are not called by the active frontend.

Feature workflows

Feature	User flow and records
Dashboard	Dashboard calls staff count, client count, job count, and aggregate totals concurrently; displays six metric cards. All four read endpoints are public at the API layer.
Clients	List -> GET all clients; create -> POST contact/business values; view/edit -> GET and PUT by client ID; status/delete actions patch or delete; ClientJobs calls filtered job query and sums payment. Client records point to Admin by adminId.
Staff and accounts	Create staff -> POST /user/addUser with role=staff and nested staffDetails, creating User and Staff documents. List/view/edit/status flows use /staff and /user endpoints. StaffJobs queries assigned jobs and can update payment status.
Admins	Create admin -> POST /user/addUser with role=admin and nested adminDetails; list reads admin role users; view populates profile; edit updates admin profile or password. The active UI does not use legacy admin register/login.
Jobs	Create form loads active clients/staff, posts assignment and hours/rates; API verifies references and computes totals. List loads all jobs or staff's jobs and filters; detail/edit reads and updates. Status, delete, and time clock actions are separate API calls.
Time tracking	Staff presses Sign In: API assigns staff ID, stores signInTime/isSignIn. Sign Out stores signOffTime/isSignOff. The API does not verify the caller identity or assignment and does not calculate actual hours.
Completed jobs and payment summary	Completed list filters jobStatus=Completed; payment summary totals orgTotal, staffPayTotal, and their difference. Its payment/status actions call public job endpoints.
Invoices	Create/edit forms load active clients and submit invoice fields; list supports sendDate filtering, client population, status update, delete, and browser PDF export; detail shows populated Client.
Profile	EditProfile derives User ID from the JWT, reads user data, updates userName/email, and uploads/removes profile image through a separate hosted API. ChangePw calls updateUserPassword with current and new password.
Password reset	Backend functions intend to issue and verify an OTP and set a Staff password, but no reset routes are registered and resetPassword refers to an undefined model. There is no working frontend reset flow in App.js.

Authentication assessment

The main authentication mechanism has recognizable elements: bcrypt password comparison, signed JWTs, an expiration, and backend role lookups from the User collection. The server middleware uses the current database role rather than trusting the role claim alone. These controls only protect endpoints where the middleware is explicitly attached.

Observed control

Control	Assessment
Credential storage	User password is bcrypt-hashed on creation and compared on login. Legacy Admin/Staff paths are inconsistent with their schemas.
Token contents / lifetime	JWT contains user ID and role, expires after 259200 seconds (3 days). No refresh or server-side revocation path is present.
Browser storage	Token is persisted in localStorage, where same-origin script can read it. Logout clears browser storage only.
Frontend route gate	PrivateRoute performs local decode/expiry/role checks. It does not validate the signature; security depends on backend middleware.
Server gate	roleAuthentication checks signature, User existence and allowed role. It does not check User.status.
Coverage	API protection is inconsistent: all invoice endpoints and most client/staff maintenance endpoints are guarded; many user and job endpoints are public.
Bootstrap / account creation	Public addsuperAdmin allows creation of a privileged account and is a direct privilege-escalation path.
Validation / abuse controls	Source shows no rate limiting, login throttling, password reset route wiring, CSRF strategy, or centralized API auth client.

Authentication flow diagram

Browser	API / database
Login form -> POST /user/loginUser {email,password}	User lookup -> bcrypt compare -> jwt.sign({_id,role}, SECRET, exp=3 days)
Store token in localStorage; navigate /dashboard	Response returns token; no cookie or refresh token
PrivateRoute decodes token; checks exp/role	This check is client-only and can be bypassed by direct requests
Axios call with Bearer token where present	roleAuthentication verifies signature -> loads User role -> allowlist -> controller
Logout clears localStorage	No token revocation or audit event

Main identity dependency: middleware searches User by the JWT _id. Tokens created by the legacy Admin or Staff login functions carry IDs from their respective collections, so they will generally fail the User lookup. The frontend currently uses the User login endpoint, but the legacy routes remain reachable.

Security and reliability findings

Prioritized findings

Severity	Finding and evidence	Impact
Critical	Unauthenticated POST /api/user/addsuperAdmin (backend/routes/userRoute.js; userController.addsuperAdmin). It accepts role=superAdmin and creates the account.	Any network caller able to reach the API can create a privileged identity and obtain a role-bearing token.
Critical	Hard-coded email service credential in backend/controllers/staffController.js sendEmail(). The password reset code is not mounted, but the credential literal is still in source.	Treat as exposed: rotate the mail credential, remove it from source/history as appropriate, and use managed secret configuration.
High	Sensitive user APIs have no route middleware: getAllUsers, deleteUser, updateUserPassword, updateStaff, updateAdmin, userUpdate, getUser (backend/routes/userRoute.js).	Unauthenticated reads or ID-based account/profile changes; getAllUsers returns password hashes.
High	Most job endpoints are public, including getAllJobs, deleteJobById, updateStatus, signInJob, signOffJob, getStaffJobsbyId, getFilteredJobs and paymentJob (backend/routes/jobRoute.js).	Anyone can enumerate or alter job data, timestamps, assigned staff, payment state, and status.
High	Public counts expose operational metrics; /client/getAllClientCount and /staff/getAllStaffCount are unguarded. Dashboard uses these APIs without a token.	Unauthenticated access reveals business volumes; job totals endpoint also exposes aggregate financials.

Severity	Finding and evidence	Impact
High	Inactive users are not rejected in loginUser or roleAuthentication; middleware only checks existence and role.	Changing a user to inactive does not reliably revoke access or stop new sessions.
High	Client/Job required adminId fields reference Admin, but addClient/addJob put req.userId (a User ID) into them. Staff's legacy create path reads req.adminId.id even though middleware sets req.userId and req.userRole.	Owner references can point at the wrong collection; population and ownership queries become unreliable, while the legacy staff endpoint can fail.
High	Separate Admin and Staff login/create implementations do not agree with their Mongoose schemas: Admin schema omits email/password/role; Staff schema omits email/password/role. Legacy tokens use Admin/Staff IDs, while middleware reloads User.	Legacy account setup/login flows are likely nonfunctional or create identity records unusable by the main auth middleware.
Medium	JWT is stored in localStorage for three days; frontend logout only clears local values.	Script injection can steal bearer tokens; logout cannot revoke a copied token.
Medium	No endpoint-wide validation or abuse controls are visible: open CORS, no rate limiting, and controllers accept ID-based mutations with limited validation.	Increases exposure to brute force, malformed requests, and cross-origin API use; severity depends on deployment/network controls.
Medium	Staff password reset handlers are not routed; resetPassword references undefined PassangerModel; controller also references missing imports in its reset helpers.	Password recovery cannot be relied on and may throw at runtime if later exposed.
Medium	paymentJob always sets jobStatus to Cancelled after changing paymentStatus; UI label suggests this is a payment status operation.	A payment update can unexpectedly cancel a job and distort job completion reporting.
Low	signInJob/signOffJob respond with data but frontend reads response.data.job; getInvoice reads field names absent from the schemas.	Some client-side refresh and invoice display logic may show stale or missing values.

Recommended order of remediation

- 1. Remove the public superAdmin creation route from deployed access; provision the first privileged account through a controlled one-time process.
- 2. Add server-side authentication and role allowlists to every user/job/count endpoint that returns business data or mutates records. Add ownership checks where records belong to a caller.
- 3. Rotate the hard-coded mail credential and remove it from tracked source. Keep secrets only in a managed environment or secret store.
- 4. Choose User as the single authentication principal. Align profile references and remove or repair Admin/Staff legacy authentication flows and their routes.
- 5. Reject inactive users at login and on every authenticated request; add a session revocation/version mechanism if immediate logout or disable is required.
- 6. Add request validation, login throttling, rate limits, and structured audit logs. Review CORS and production API URL configuration.
- 7. Repair password recovery, payment state transitions, response shapes, and schema/controller field mismatches; then add auth and workflow tests.

These are remediation recommendations only. No application source code was changed as part of this report.

Document relationship map

From	Relationship	To / note
User	optional refs	Admin, Staff, SuperAdmin profile documents
Staff	user ref	User; active UI creates Staff detail from User.addUser
Client	adminId ref	Admin; controller currently supplies req.userId from User, which is a mismatched collection reference
Job	client ref	Client
Job	assignedStaff ref	Staff
Job	adminId ref	Admin; controller currently supplies req.userId from User
Invoice	client ref	Client
Admin / Staff / SuperAdmin	user ref	User principal

Local runtime setup is split: frontend uses Create React App, backend uses Express with nodemon. backend/index.js

expects PORT, MONGO_URI, and SECRET; email support expects USER_EMAIL and USER_PASS. The frontend API URL is hardcoded to localhost:4000/api. Confirm a deployment-specific URL and secret management before a production build.

Static review caveats: environment files were not opened; no values from them are reflected here. No server was started, no database was contacted, no routes were exercised, and no tests or builds were run. Findings describe the checked-in source paths, not external infrastructure settings.